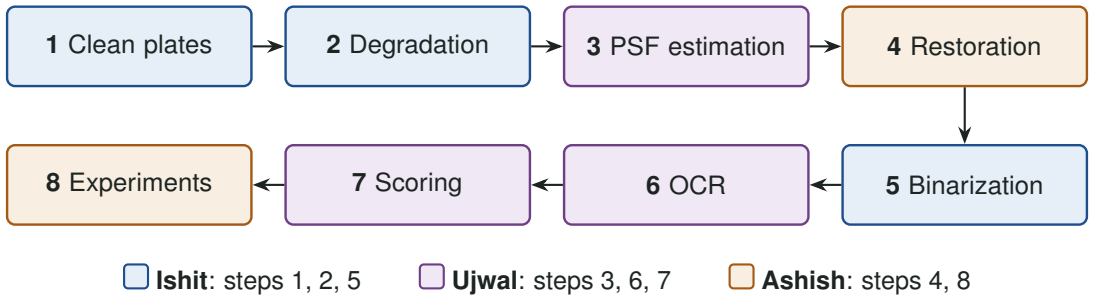
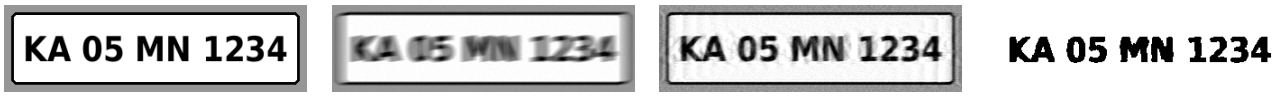
Work Division

License plate restoration · CS7.404 DIP project · team reference

Each of us owns a few steps of the pipeline. The steps are numbered in the order an image moves through them, and each person's tasks below follow that order. "Hands over" says what you give to whom when your step is done.





Step 1: clean plate | Step 2: blurred + noisy | Step 4: restored (Wiener) | Step 5: binarized

All images in this document come from one test run of the pipeline on a synthetic plate (motion blur 15 px, noise at 30 dB).

Agree on these in week 1, before coding

- Images are grayscale `float32` in $[0, 1]$. Convert to 8-bit only right before OCR.
- PSFs are odd-sized, centred, and sum to 1. Angle is measured in degrees, counter-clockwise from the x-axis.
- Function names and inputs/outputs for each step, so everyone can test with dummy versions of the others' parts.

Who passes what to whom

From	What	To
Ishit (steps 1, 2)	clean plates, true text, degraded images, true PSF	Ujwal (step 3), Ashish (step 4), Ujwal (step 7)
Ujwal (step 3)	estimated PSF	Ashish (step 4)
Ashish (step 4)	restored images	Ishit (step 5), Ujwal (step 7)
Ishit (step 5)	binary plate, character crops	Ujwal (step 6)
Ujwal (steps 6, 7)	OCR text, confidence, scores, outcome	Ashish (step 8)
Ashish (step 8)	results table and plots	everyone (report)

Ishit Steps 1, 2, 5

STEP 1 Clean plates

1. Write a plate renderer: random valid Indian numbers (format `AA 00 AA 0000`), black text on a white plate, 320×80 px with an 8 px grey margin. Make about 600.
2. Write a CCPD loader: read the four plate corners from each filename, straighten the plate to 240×76 px and add the same margin. About 400 from `ccpd_base`, 50 from `ccpd_blur`.
3. Save every plate with its true text in one place (e.g. an `.npz` file plus a `.csv` list).



Rendered synthetic plate with known text.

Hands over: clean plates and their true text → everyone.

STEP 2 Degradation

1. Motion PSF from a blur length and angle; defocus PSF (a disk) from a radius.
2. Blur the plate using the FFT (pad the edges first), downsample by 1 or 2, then add noise at a target BSNR.
3. Loop over the grid: blur length 0–21 px, defocus radius 0–5 px, noise at 40, 30, 25, 20 dB. Log every parameter and the random seed.

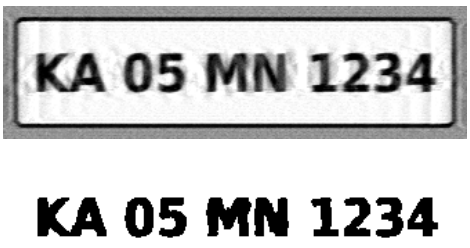


Motion PSF (15 px, 8°) and the plate after blur and noise.

Hands over: degraded image + true PSF → Ujwal (step 3) and Ashish (step 4).

STEP 5 Binarization

1. Otsu thresholding from scratch, to turn the restored plate into black text on white.
2. Erosion and dilation from scratch; build opening (removes specks) and closing (joins broken strokes) from them. Check against OpenCV.
3. Find connected components, keep the character-shaped ones, and sort them left to right.



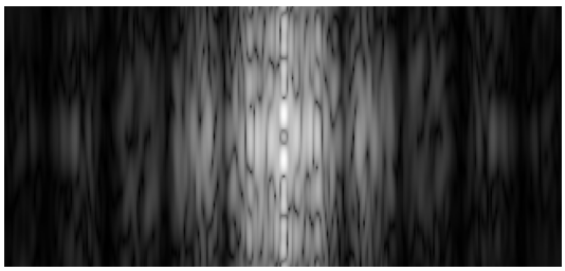
Restored plate (top) after Otsu + opening (bottom).

Hands over: binary plate + character crops → Ujwal (step 6).

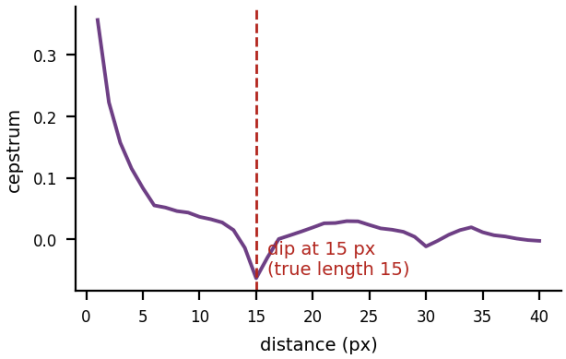
Ujwal Steps 3, 6, 7

STEP 3 PSF estimation (blind)

1. Subtract the mean, apply a Hann window and take the log magnitude spectrum. Motion blur shows up as parallel dark stripes.
2. **Angle:** rotate the spectrum and find the direction in which the stripes line up.
3. **Length:** compute the cepstrum; it has a negative dip at the blur length.
4. Build the estimated PSF. Keep the true PSF too ("oracle") so the two can be compared. Log the estimated length and angle.



Spectrum of a horizontally blurred plate: vertical dark stripes.



Cepstrum along the blur direction: the dip lands on the true length.

Hands over: estimated PSF → Ashish (step 4).

STEP 6 OCR

1. Run Tesseract on the whole binary plate (`-psm 7`, letters and digits only) and on each character crop (`-psm 10`).
2. Record the text read and the confidence score.
3. First check that clean plates read at 95% or better; fix preprocessing (size, border) before going further.

Hands over: read text + confidence → step 7.

STEP 7 Scoring

1. PSNR and SSIM from scratch (check against scikit-image).
2. Character error rate using edit distance against the true text.
3. Label every result as one of the three outcomes on the right, using a confidence threshold of 70.

Correct: text matches

Honest failure: wrong, confidence $<$ 70

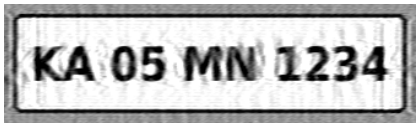
Silent failure: wrong, confidence $\geq$ 70

Hands over: one result row per run (scores + outcome) → Ashish (step 8).

Ashish — Steps 4, 8

STEP 4 Restoration

1. Helpers: turn a PSF into its frequency response (pad to image size, shift the centre to the corner), and taper the image edges to reduce ringing.
2. Four filters, each taking (degraded image, PSF, parameter) and returning the restored image: inverse filter (threshold ε), Wiener (noise constant K), constrained least squares (γ, Laplacian), Richardson–Lucy (number of iterations).
3. For Richardson–Lucy, save snapshots at several iterations from a single run.



Inverse filter, PSNR 18.1 dB



Wiener (K=0.01), PSNR 18.9 dB

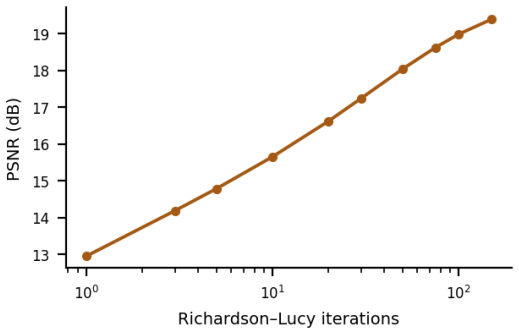


Richardson–Lucy (30 it.), PSNR 17.3 dB

Hands over: restored image → Ishit (step 5) and Ujwal (step 7, for PSNR/SSIM).

STEP 8 Experiments

1. Write the runner: loop over plates × degradations × filters × parameter values, once with the true PSF and once with the estimated one.
2. Cache results to disk so a crash doesn't mean rerunning everything. Use 4 CPU cores; no GPU is needed.
3. Write all results to one `results.csv`, then make the plots: PSNR and OCR accuracy vs. parameter, true vs. estimated PSF, outcome bars, and a blur × noise heatmap.



Example sweep on the test plate: PSNR keeps rising with more iterations. The real plot adds OCR accuracy on the same axis.

Hands over: final tables and plots → everyone, for the report.